ACADEMIC PROJECT PRESENTATION

Extracting Stock Data Using Web Scraping

A Python-based workflow for collecting, cleaning, and visualizing financial data

Student: Tasmia| Date: 10/16/2026

Project Overview

A structured introduction to automated financial data collection

Problem

Manual collection of stock and revenue information can be slow and difficult to repeat.

Approach

Use Python to retrieve HTML, identify tables, extract records, and organize the results.

Outcome

Create structured datasets that can be inspected, cleaned, and visualized.

Case webpages: course-provided Amazon and Netflix HTML pages.

Objectives

What the project aims to demonstrate

- Retrieve webpage content with Python’s requests library.
- Parse HTML and locate relevant tables using BeautifulSoup.
- Convert extracted records into pandas DataFrames.
- Clean dates and numeric values for analysis.
- Prepare visualizations of stock-price and revenue trends.
- Evaluate practical benefits and limitations of web scraping.

Tools and Technologies

Each library performs a distinct role in the workflow

Python

Core programming language

requests

Downloads webpage HTML

BeautifulSoup

Parses HTML and finds tables

pandas

Structures and cleans data

Matplotlib

Visualizes data over time

Methodology and Workflow

From raw webpage content to analysis-ready data



Request

Download HTML

Parse

Read HTML structure

Extract

Select relevant tables

Clean

Fix dates and values

Analyze

Inspect patterns

Visualize

Plot trends

Quality checks should be applied before interpreting or presenting the results.

Step 1: Retrieve the Webpage

Example using the course-provided Netflix HTML page

```
import requests

url = (
    "https://cf-courses-data.s3.us.cloud-object-storage."
    "appdomain.cloud/IBMDeveloperSkillsNetwork/"
    "PY0220EN-SkillsNetwork/labs/project/"
    "netflix_data_webpage.html"
)

response = requests.get(url, timeout=30)
response.raise_for_status()
html_data = response.text
```

requests.get()

Sends an HTTP request to the URL.

response.text

Stores the returned HTML as a string.

Step 2: Parse HTML and Find Tables

BeautifulSoup makes webpage structure easier to inspect

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_data, "html.parser")
tables = soup.find_all("table")

print("Number of tables:", len(tables))
```

Parse

Convert HTML text into a navigable structure.

Inspect

Identify which table contains the required fields.

Important: select tables by their actual contents; do not assume a fixed table index.

Data Extraction and Preparation

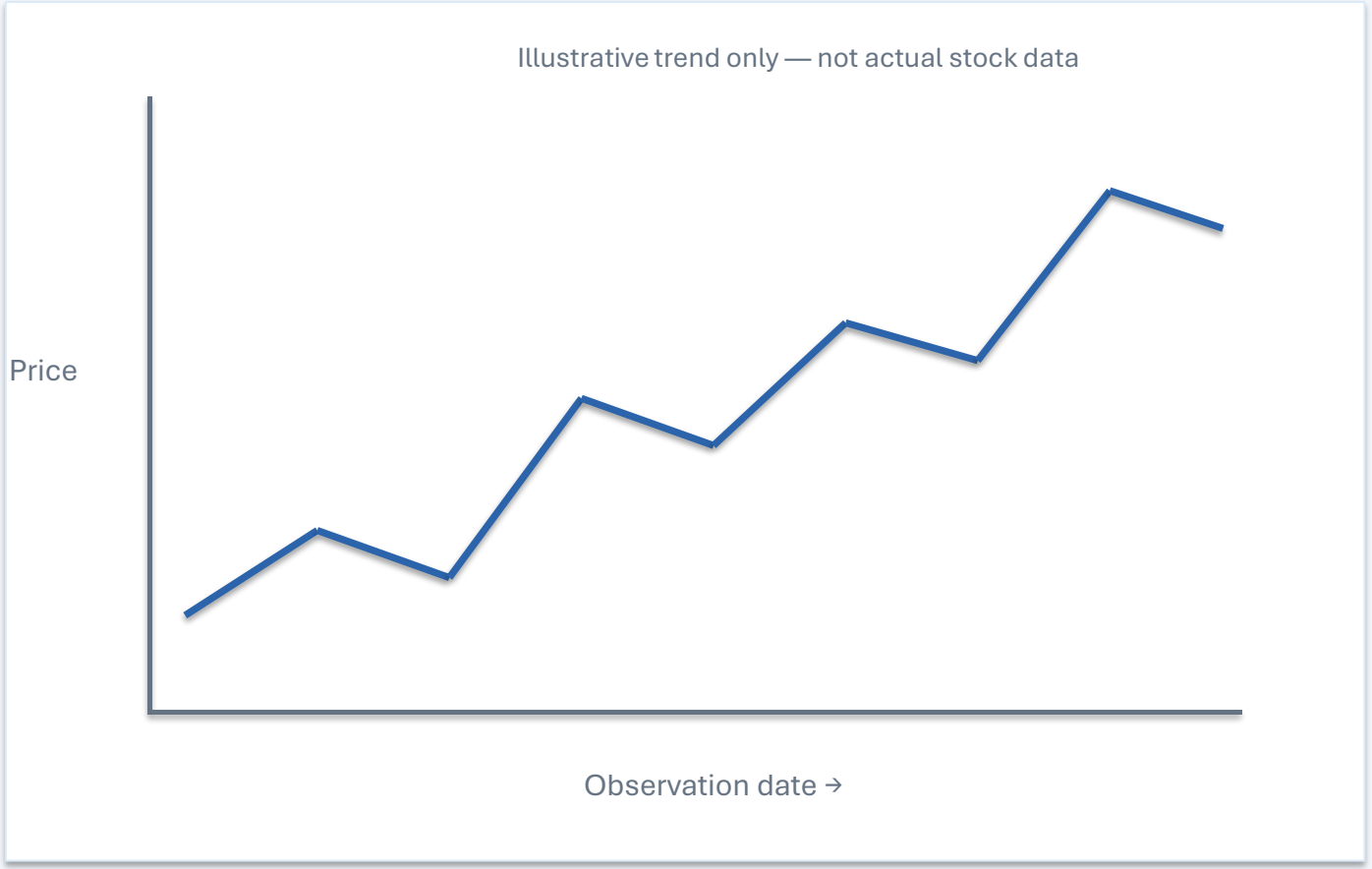
Convert scraped records into analysis-ready datasets

- Identify the table containing stock-price or revenue information.
- Extract table headers and row values into lists or dictionaries.
- Create a pandas DataFrame with clearly named columns.
- Convert dates to datetime and prices/revenue to numeric values.
- Check missing values, duplicates, unexpected formats, and chronological order.

The exact extraction code depends on the HTML structure of the selected webpage.

Analysis and Visualization

Visual summaries can reveal changes over the observation period



Stock prices

Use a line chart to examine closing-price movements.

Revenue

Plot quarterly revenue by reporting date.

Interpret trends cautiously; correlation does not prove causation.

Results and Interpretation

Report findings only after executing and validating the code

The workflow produces HTML content that can be parsed for relevant tables.

Extracted records can be organized into DataFrames for inspection and cleaning.

Charts can summarize historical price and revenue patterns.

Stock prices and company revenue represent different aspects of performance.

Numerical findings must be supported by the actual extracted data.

Strengths and Limitations

A balanced evaluation of the approach

Strengths

- Automates repetitive data collection
- Uses accessible Python tools
- Produces structured datasets
- Supports visual exploration

Limitations

- HTML changes can break extraction
- Source data may be incomplete
- Course pages may not be real-time
- Descriptive charts do not establish causality

Recommendations for Future Work

How the project could be made more robust

Add error handling for network and parsing failures.

Validate table headers, row counts, dates, and numeric values.

Document data-cleaning decisions and missing-value treatment.

Compare companies over a consistent reporting period.

Consider an authorized financial data API for regularly updated data.

Respect website terms, access restrictions, and applicable data-use rules.

Conclusion

Web scraping provides a practical method for converting webpage-based financial information into structured data. By combining requests, BeautifulSoup, pandas, and visualization tools, the project demonstrates an end-to-end workflow for data collection and preparation.

The reliability of conclusions depends on validating the extracted data and interpreting stock-price and revenue trends within their proper context.

References

Documentation and course resources

Python Software Foundation. Python Documentation — https://docs.python.org/3/

Requests Documentation — https://requests.readthedocs.io/

Beautiful Soup Documentation — https://www.crummy.com/software/BeautifulSoup/bs4/doc/

pandas Documentation — https://pandas.pydata.org/docs/

Matplotlib Documentation — https://matplotlib.org/stable/

IBM Developer Skills Network. Course-provided Amazon and Netflix HTML webpages.